

Agent Foundations - Write-Up

Week 5, Day 1 | Raw Python Agent (No Framework)

Endpoint Used

This project was implemented using the OpenAI-compatible endpoint provided by NetixSol (<https://llm.netixsol.com/v1>). Although the assignment references the Anthropic API, the provided endpoint follows the OpenAI Chat Completions format. The concepts remain the same, with Anthropic's `tool_use` and `tool_result` corresponding to OpenAI's `tool_calls` and `tool` messages.

ReAct Loop

The agent follows the **ReAct (Reason → Act → Observe)** pattern.

1. **Reason:** The model analyzes the user's request.
2. **Act:** If additional information or computation is required, it requests a tool.
3. **Observe:** Python executes the tool and returns the result to the model.
4. The process repeats until the model produces a final answer or reaches the maximum iteration limit.

```
while not done and iterations < max_iterations:
    response = model(messages)

    if response.tool_calls:
        execute_tool()
        append_tool_result()
    else:
        done = True
```

A `max_iterations` safeguard was added to prevent infinite loops.

The agent maintains two kinds of memory:

- **Conversation memory:** the message history sent back to the model on every API call.
- **Working memory:** a Python scratchpad used to track iteration count and executed tool calls for debugging and control flow.

Tool Schemas Used

Three tools were implemented:

- **calculator(expression):** Safely evaluates arithmetic expressions using Python's AST module.
- **get_weather(city):** Returns weather information from a small predefined dataset.
- **read_text_file(path):** Reads the contents of a local text file.

Each tool was defined using a JSON schema containing its name, description, input parameters, and required fields. Tool descriptions are important because the model relies entirely on them to decide when and how to call a tool.

Failure Modes Observed

Three scenarios were tested:

Scenario A – Ambiguous Request

Input: *"Can you book it for me?"*

The agent asked for clarification instead of making assumptions.

Scenario B – Tool Error

Input: *"What is 10 / 0?"*

The model answered directly that division by zero is undefined instead of calling the calculator tool.

Scenario C – Undefined Capability

Input: *"Please send an email to my team about the project delay."*

Since no email tool existed, the model offered to draft the email instead of attempting an unsupported action.

Common failure modes and their mitigations include:

Failure Mode	Mitigation
Infinite loop	Limit execution using <code>max_iterations</code> .
Hallucinated tool calls	Verify that requested tools exist before execution.
Invalid tool arguments	Validate JSON arguments before calling a tool.
Tool execution errors	Return structured error messages instead of crashing.
Ambiguous requests	Ask clarifying questions when needed.
High API cost	Limit iterations and use smaller models for simple tasks.

Why Frameworks Exist

Building the agent manually demonstrated how tool calling, conversation memory, logging, and the ReAct loop work internally. Frameworks such as **LangChain**, **LangGraph**, and **CrewAI** automate these repetitive components, making it easier to build larger AI applications while allowing developers to focus on application logic instead of infrastructure.